

16. Compare different types Classification Algorithms and evaluate their performance.

CODING:

```
import os
import warnings

# 1. Prevent joblib/loky CPU core lookup warning on Windows 11
os.environ["LOKYMAYCPUCOUNT"] = "4"
warnings.filterwarnings('ignore', category=UserWarning)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

#
# DATASET DEFINITIONS & TEST SAMPLES
#

# Dataset 1: Student Placement Prediction
d1data = {
    'CGPA': [9.2, 8.8, 8.5, 7.2, 6.8, 9.0, 7.5, 8.6, 6.5, 8.9],
    'Aptitude Score': [90, 85, 80, 72, 68, 92, 75, 84, 60, 88],
    'Communication Score': [88, 82, 79, 70, 65, 90, 74, 81, 58, 86],
    'Placement': ['Placed', 'Placed', 'Placed', 'Not Placed', 'Not Placed',
                  'Placed', 'Not Placed', 'Placed', 'Not Placed', 'Placed']
}
d1sample = pd.DataFrame([8.7, 86, 84], columns=['CGPA', 'Aptitude Score', 'Communication Score'])

# Dataset 2: Loan Approval Prediction
d2data = {
    'Income': [8, 7, 6, 4, 3, 9, 5, 8, 4, 7],
    'Credit Score': [780, 760, 730, 620, 590, 810, 650, 770, 610, 750],
    'Loan Amount': [5, 4, 5, 8, 9, 4, 7, 5, 8, 6],
    'Loan Status': ['Approved', 'Approved', 'Approved', 'Rejected', 'Rejected',
                   'Approved', 'Rejected', 'Approved', 'Rejected', 'Approved']
}
d2sample = pd.DataFrame([7, 760, 5], columns=['Income', 'Credit Score', 'Loan Amount'])

# Dataset 3: Disease Diagnosis
d3data = {
    'Temperature': [39.0, 38.5, 37.0, 39.2, 36.8, 38.8, 37.2, 39.1, 36.7, 38.9],
    'Heart Rate': [110, 108, 78, 115, 75, 112, 80, 114, 74, 111],
    'Oxygen Level': [94, 95, 99, 93, 98, 94, 98, 92, 99, 93],
```

```

'Disease': ['Positive', 'Positive', 'Negative', 'Positive', 'Negative',
            'Positive', 'Negative', 'Positive', 'Negative', 'Positive']
}
d3sample = pd.DataFrame([[38.7, 109, 94]], columns=['Temperature', 'Heart Rate', 'Oxygen Level'])

# Dataset 4: Employee Promotion Prediction
d4data = {
    'Experience': [10, 8, 7, 3, 2, 9, 4, 8, 2, 6],
    'Performance Score': [95, 90, 88, 65, 60, 93, 70, 89, 58, 85],
    'Training Hours': [50, 45, 40, 20, 18, 48, 25, 42, 15, 38],
    'Promotion': ['Promoted', 'Promoted', 'Promoted', 'Not Promoted', 'Not Promoted',
                  'Promoted', 'Not Promoted', 'Promoted', 'Not Promoted', 'Promoted']
}
d4sample = pd.DataFrame([[7, 90, 44]], columns=['Experience', 'Performance Score', 'Training
Hours'])

# Dataset 5: Vehicle Classification
d5data = {
    'Engine Capacity': [100, 125, 150, 800, 1000, 1200, 110, 900, 140, 1300],
    'Weight': [95, 110, 125, 850, 950, 1050, 100, 900, 120, 1100],
    'Mileage': [70, 65, 55, 22, 20, 18, 68, 21, 58, 17],
    'Vehicle Type': ['Bike', 'Bike', 'Bike', 'Car', 'Car', 'Car', 'Bike', 'Car', 'Bike', 'Car']
}
d5sample = pd.DataFrame([[135, 118, 60]], columns=['Engine Capacity', 'Weight', 'Mileage'])

# Compile all experiments
experiments = [
    ("1: Student Placement Prediction", d1data, 'Placement', d1sample),
    ("2: Loan Approval Prediction", d2data, 'Loan Status', d2sample),
    ("3: Disease Diagnosis", d3data, 'Disease', d3sample),
    ("4: Employee Promotion Prediction", d4data, 'Promotion', d4sample),
    ("5: Vehicle Classification", d5data, 'Vehicle Type', d5sample),
]

#
# EXECUTION & COMPARISON
#

for title, dataset, targetcol, newsample in experiments:
    print(""" * 50)
    print(f'Dataset {title}')
    print(""" * 50)

    # Load dataset
    df = pd.DataFrame(dataset)
    X = df.drop(columns[targetcol])
    y = df[targetcol]

```

```

# Train/Test Split (70% train, 30% test)
Xtrain, Xtest, ytrain, ytest = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train Models
dtmodel = DecisionTreeClassifier(random_state=42)
dtmodel.fit(Xtrain, ytrain)

knnmodel = KNeighborsClassifier(n_neighbors=3)
knnmodel.fit(Xtrain, ytrain)

# Predictions
dtpredictions = dtmodel.predict(Xtest)
knnpredictions = knnmodel.predict(Xtest)

# Calculate Accuracy
dtaccuracy = accuracy_score(ytest, dtpredictions)
knnaccuracy = accuracy_score(ytest, knnpredictions)

# Display Results
print(f'Decision Tree Accuracy: {dtaccuracy:.2f}')
print(f'KNN Accuracy: {knnaccuracy:.2f}')

if dtaccuracy > knnaccuracy:
    print("Result: Decision Tree performed better.")
elif knnaccuracy > dtaccuracy:
    print("Result: KNN performed better.")
else:
    print("Result: Both algorithms achieved the same accuracy.")

# Predict New Sample
dtsamplepred = dtmodel.predict(newsample)[0]
knnsamplepred = knnmodel.predict(newsample)[0]

print("\n New Sample Prediction ")
print(f'Decision Tree Prediction: {dtsamplepred}')
print(f'KNN Prediction: {knnsamplepred}\n')

```

OUTPUT:

Dataset 1: Student Placement Prediction

Decision Tree Accuracy: 1.00

KNN Accuracy: 1.00

Result: Both algorithms achieved the same accuracy.

New Sample Prediction

Decision Tree Prediction: Placed

KNN Prediction: Placed

Dataset 2: Loan Approval Prediction

Decision Tree Accuracy: 1.00

KNN Accuracy: 1.00

Result: Both algorithms achieved the same accuracy.

New Sample Prediction

Decision Tree Prediction: Approved

KNN Prediction: Approved

Dataset 3: Disease Diagnosis

Decision Tree Accuracy: 1.00

KNN Accuracy: 1.00

Result: Both algorithms achieved the same accuracy.

New Sample Prediction

Decision Tree Prediction: Positive

KNN Prediction: Positive

Dataset 4: Employee Promotion Prediction

Decision Tree Accuracy: 1.00

KNN Accuracy: 1.00

Result: Both algorithms achieved the same accuracy.

New Sample Prediction

Decision Tree Prediction: Promoted

KNN Prediction: Promoted

Dataset 5: Vehicle Classification

Decision Tree Accuracy: 1.00

KNN Accuracy: 1.00

Result: Both algorithms achieved the same accuracy.

New Sample Prediction

Decision Tree Prediction: Bike

KNN Prediction: Bike

17. Implement Mobile Price Prediction using appropriate machine learning algorithm

CODE:

```
import os
```

```
import warnings
```

```
# 1. Prevent joblib/loky CPU core lookup warning on Windows
```

```
os.environ["LOKYMAYCPUCOUNT"] = "4"
```

```
warnings.filterwarnings('ignore', category=UserWarning)
```

```
import pandas as pd
```

```
from sklearn.linear_model import LinearRegression
```

```
#
```

```
# DATASETS & NEW TEST SAMPLES DEFINITION
```

```
#
```

```
# Dataset 1: Mobile Specifications vs Price
```

```
d1data = {
```

```
    'RAM (GB)': [4, 4, 6, 6, 8, 8, 12, 12, 16, 16],
```

```
    'Storage (GB)': [64, 128, 128, 256, 128, 256, 256, 512, 512, 1024],
```

```
    'Camera (MP)': [12, 16, 48, 50, 64, 64, 108, 108, 200, 200],
```

```
    'Price (₹)': [12000, 15000, 18000, 22000, 25000, 30000, 38000, 45000, 60000, 75000]
```

```
}
```

```
d1sample = pd.DataFrame([[8, 256, 108]], columns=['RAM (GB)', 'Storage (GB)',
```

```
'Camera (MP)'])
```

Dataset 2: Processor Performance vs Mobile Price

```
d2data {  
  'Processor Speed (GHz)': [1.8, 2.0, 2.2, 2.4, 2.6, 2.8, 3.0, 3.2, 3.4, 3.6],  
  'RAM (GB)': [4, 4, 6, 6, 8, 8, 12, 12, 16, 16],  
  'Battery (mAh)': [4000, 4500, 5000, 5000, 5500, 6000, 6000, 6500, 7000,  
5000],  
  'Price (₹)': [10000, 12000, 17000, 21000, 26000, 32000, 40000, 50000,  
62000, 76000]  
}  
d2sample pd.DataFrame([[2.9, 8, 6000]], columns['Processor Speed (GHz)',  
'RAM (GB)', 'Battery (mAh)'])
```

Dataset 3: Display Features vs Mobile Price

```
d3data {  
  'Screen Size (inches)': [6.1, 6.2, 6.4, 6.5, 6.6, 6.7, 6.8, 6.9, 7.0, 7.1],  
  'Refresh Rate (Hz)': [60, 90, 90, 120, 120, 120, 144, 144, 165, 165],  
  'Battery (mAh)': [4000, 4500, 5000, 5000, 5500, 6000, 6000, 6500, 7000,  
7000],  
  'Price (₹)': [12000, 15000, 18000, 23000, 28000, 34000, 42000, 52000,  
65000, 78000]  
}  
d3sample pd.DataFrame([[6.7, 144, 6000]], columns['Screen Size (inches)',  
'Refresh Rate (Hz)', 'Battery (mAh)'])
```

Dataset 4: Premium Smartphone Dataset

```
d4data {  
  'RAM (GB)': [4, 6, 8, 8, 12, 12, 16, 16, 18, 24],  
  'Processor Cores': [6, 8, 8, 8, 8, 8, 8, 8, 8, 8],  
  'Internal Storage (GB)': [64, 128, 128, 256, 256, 512, 512, 1024, 1024, 1024],  
  'Price (₹)': [14000, 19000, 26000, 32000, 42000, 52000, 65000, 82000,  
95000, 120000]  
}  
d4sample pd.DataFrame([[12, 8, 512]], columns['RAM (GB)', 'Processor  
Cores', 'Internal Storage (GB)'])
```

Dataset 5: Overall Mobile Performance Dataset

```
d5data {
```

```

    'Antutu Score': [250000, 320000, 420000, 520000, 620000, 720000, 820000,
920000, 1020000, 1120000],
    'Battery (mAh)': [4000, 4500, 5000, 5000, 5500, 6000, 6000, 6500, 7000,
7000],
    'Camera (MP)': [48, 50, 64, 64, 108, 108, 200, 200, 200, 200],
    'Price (₹)': [15000, 18000, 24000, 30000, 38000, 46000, 58000, 70000,
85000, 100000]
}
d5sample = pd.DataFrame([[750000, 6000, 108]], columns=['Antutu Score',
'Battery (mAh)', 'Camera (MP)'])

```

```

# Group experiments together

```

```

experiments [
    ("Dataset 1: Mobile Specifications vs Price", d1data, 'Price (₹)', d1sample),
    ("Dataset 2: Processor Performance vs Mobile Price", d2data, 'Price (₹)',
d2sample),
    ("Dataset 3: Display Features vs Mobile Price", d3data, 'Price (₹)', d3sample),
    ("Dataset 4: Premium Smartphone Dataset", d4data, 'Price (₹)', d4sample),
    ("Dataset 5: Overall Mobile Performance Dataset", d5data, 'Price (₹)',
d5sample)
]

```

```

#

```

```

# PROCEDURE EXECUTION

```

```

#

```

```

for title, data, targetcol, newsample in experiments:

```

```

    print(""" * 60)
    print(title)
    print(""" * 60)

```

```

# Step 2: Create Dataset (Separate X and y)

```

```

df = pd.DataFrame(data)
X = df.drop(columns=[targetcol])
y = df[targetcol]

```

```

# Step 1 & 3: Import & Train Linear Regression Model

```

```

model = LinearRegression()

```

```
model.fit(X, y)
```

```
# Step 4: Predict Mobile Price for the new sample  
predictedprice model.predict(newsample)[0]
```

```
# Step 5: Display Result  
print(f'Sample Input:\n{newsample.tostring(indexFalse)}')  
print(f'\nPredicted Mobile Price: ₹ {predictedprice:,.2f}\n')
```

OUTPUT:

Dataset 1: Mobile Specifications vs Price

Sample Input:

RAM (GB)	Storage (GB)	Camera (MP)
8	256	108

Predicted Mobile Price: ₹31,317.27

Dataset 2: Processor Performance vs Mobile Price

Sample Input:

Processor Speed (GHz)	RAM (GB)	Battery (mAh)
2.9	8	6000

Predicted Mobile Price: ₹34,122.16

Dataset 3: Display Features vs Mobile Price

Sample Input:

Screen Size (inches)	Refresh Rate (Hz)	Battery (mAh)
6.7	144	6000

Predicted Mobile Price: ₹42,603.34

Dataset 4: Premium Smartphone Dataset

Sample Input:

RAM (GB)	Processor Cores	Internal Storage (GB)
12	8	512

Predicted Mobile Price: ₹53,109.22

Dataset 5: Overall Mobile Performance Dataset

Sample Input:

Antutu Score	Battery (mAh)	Camera (MP)
750000	6000	108

Predicted Mobile Price: ₹55,470.33

18. Implement Perceptron based IRIS classification

CODE:

```
import os
```

```
import warnings
```

```
# 1. Prevent joblib/loky CPU core lookup warning on Windows
```

```
os.environ["LOKYMAYCPUCOUNT"] "4"
```

```
warnings.filterwarnings('ignore', category=UserWarning)
```

```
import pandas as pd
```

```
from sklearn.linear_model import Perceptron
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracyscore
```

```
#
```

```
# DATASET DEFINITIONS & TEST SAMPLES
```

```
#
```

```
# Dataset 1: Fruit Classification
```

```
d1data {
```

```
    'Weight (g)': [150, 160, 145, 120, 125, 130, 155, 118, 148, 122],
```

```

'Diameter (cm)': [7.5, 7.8, 7.3, 5.5, 5.8, 6.0, 7.6, 5.4, 7.4, 5.7],
'Sweetness (%)': [90, 88, 91, 70, 72, 74, 89, 69, 92, 71],
'Fruit': ['Apple', 'Apple', 'Apple', 'Orange', 'Orange', 'Orange', 'Apple',
'Orange', 'Apple', 'Orange']
}
d1sample pd.DataFrame([[152, 7.5, 90]], columns['Weight (g)', 'Diameter
(cm)', 'Sweetness (%)'])

```

Dataset 2: Student Placement Classification

```

d2data {
  'CGPA': [9.3, 8.9, 8.6, 7.3, 6.9, 9.1, 7.4, 8.7, 6.6, 8.8],
  'Aptitude Score': [92, 87, 84, 74, 69, 91, 75, 85, 64, 88],
  'Communication Score': [90, 85, 82, 72, 67, 89, 73, 83, 60, 86],
  'Placement': ['Placed', 'Placed', 'Placed', 'Not Placed', 'Not Placed',
    'Placed', 'Not Placed', 'Placed', 'Not Placed', 'Placed']
}
d2sample pd.DataFrame([[8.8, 86, 84]], columns['CGPA', 'Aptitude Score',
'Communication Score'])

```

Dataset 3: Disease Diagnosis

```

d3data {
  'Temperature (°C)': [39.2, 38.8, 37.0, 39.1, 36.8, 38.9, 37.2, 39.3, 36.7, 38.7],
  'Heart Rate (bpm)': [112, 110, 78, 114, 76, 111, 80, 115, 74, 109],
  'Oxygen Level (%)': [93, 94, 99, 92, 98, 94, 99, 92, 98, 95],
  'Disease': ['Positive', 'Positive', 'Negative', 'Positive', 'Negative',
    'Positive', 'Negative', 'Positive', 'Negative', 'Positive']
}
d3sample pd.DataFrame([[38.9, 110, 94]], columns['Temperature (°C)', 'Heart
Rate (bpm)', 'Oxygen Level (%)'])

```

Dataset 4: Employee Promotion Classification

```

d4data {
  'Experience (Years)': [10, 9, 8, 4, 3, 11, 5, 8, 2, 7],
  'Performance Score': [95, 92, 90, 72, 68, 96, 75, 89, 60, 87],
  'Training Hours': [50, 48, 45, 28, 24, 52, 30, 44, 18, 40],
  'Promotion': ['Promoted', 'Promoted', 'Promoted', 'Not Promoted', 'Not
Promoted',
    'Promoted', 'Not Promoted', 'Promoted', 'Not Promoted', 'Promoted']
}

```

```

}
d4sample    pd.DataFrame([[8, 90, 46]], columns['Experience (Years)',
'Performance Score', 'Training Hours'])

# Dataset 5: Vehicle Classification
d5data {
    'Engine Capacity (cc)': [100, 125, 150, 800, 1000, 1200, 110, 900, 140, 1300],
    'Weight (kg)': [95, 110, 125, 850, 950, 1050, 100, 900, 120, 1100],
    'Mileage (km/L)': [70, 65, 55, 22, 20, 18, 68, 21, 58, 17],
    'Vehicle Type': ['Bike', 'Bike', 'Bike', 'Car', 'Car', 'Car', 'Bike', 'Car', 'Bike',
'Car']
}
d5sample    pd.DataFrame([[135, 118, 60]], columns['Engine Capacity (cc)',
'Weight (kg)', 'Mileage (km/L)'])

# Group experiments
experiments [
    ("Dataset 1: Fruit Classification", d1data, 'Fruit', d1sample),
    ("Dataset 2: Student Placement Classification", d2data, 'Placement',
d2sample),
    ("Dataset 3: Disease Diagnosis", d3data, 'Disease', d3sample),
    ("Dataset 4: Employee Promotion Classification", d4data, 'Promotion',
d4sample),
    ("Dataset 5: Vehicle Classification", d5data, 'Vehicle Type', d5sample)
]

#
# PROCEDURE EXECUTION
#

for title, data, targetcol, newsample in experiments:
    print(""" * 60)
    print(title)
    print(""" * 60)

# Step 2: Separate X and y
df pd.DataFrame(data)
X df.drop(columns[targetcol])

```

```

y df[targetcol]

# Step 3: Split Dataset
Xtrain, Xtest, ytrain, ytest train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Step 4: Train Perceptron Model
model = Perceptron(random_state=42)
model.fit(Xtrain, ytrain)

# Calculate Model Accuracy on Test Set
test_preds = model.predict(Xtest)
acc = accuracy_score(ytest, test_preds)
print(f"Perceptron Model Test Accuracy: {acc:.2f}")

# Step 5 & 6: Predict & Display New Sample Class
prediction = model.predict(new_sample)[0]
print("\nSample Input Features:")
print(new_sample.tostring(index=False))
print(f"\nPredicted Class: {prediction}\n")

```

OUTPUT:

Dataset 1: Fruit Classification

Perceptron Model Test Accuracy: 0.67

Sample Input Features:

Weight (g)	Diameter (cm)	Sweetness (%)
152	7.5	90

Predicted Class: Apple

Dataset 2: Student Placement Classification

Perceptron Model Test Accuracy: 0.33

Sample Input Features:

CGPA Aptitude Score Communication Score

8.8 86 84

Predicted Class: Not Placed

Dataset 3: Disease Diagnosis

Perceptron Model Test Accuracy: 1.00

Sample Input Features:

Temperature (°C) Heart Rate (bpm) Oxygen Level (%)

38.9 110 94

Predicted Class: Positive

Dataset 4: Employee Promotion Classification

Perceptron Model Test Accuracy: 1.00

Sample Input Features:

Experience (Years) Performance Score Training Hours

8 90 46

Predicted Class: Promoted

Dataset 5: Vehicle Classification

Perceptron Model Test Accuracy: 1.00

Sample Input Features:

Engine Capacity (cc) Weight (kg) Mileage (km/L)

135 118 60

Predicted Class: Bike

19. Implementation of Naive Bayes classification for Bank Loan prediction

CODE:

```
import os
import warnings

# 1. Prevent joblib/loky CPU core lookup warning on Windows
os.environ["LOKYMAYCPUCOUNT"] = "4"
warnings.filterwarnings('ignore', category=UserWarning)

import pandas as pd
from sklearn.naivebayes import GaussianNB
from sklearn.modelselection import train_test_split
from sklearn.metrics import accuracy_score

#
# DATASET DEFINITIONS & TEST SAMPLES
#

# Dataset 1: Personal Loan Approval
d1data = {
    'Annual Income (₹ Lakhs)': [12, 10, 9, 7, 6, 5, 4, 8, 3, 11],
    'Credit Score': [820, 790, 760, 700, 680, 650, 620, 730, 600, 810],
    'Loan Amount (₹ Lakhs)': [5, 6, 7, 8, 9, 10, 12, 8, 13, 5],
    'Loan Status': ['Approved', 'Approved', 'Approved', 'Approved', 'Rejected',
                    'Rejected', 'Rejected', 'Approved', 'Rejected', 'Approved']
}
d1sample = pd.DataFrame([[9, 770, 6]], columns=['Annual Income (₹ Lakhs)',
        'Credit Score', 'Loan Amount (₹ Lakhs)'])

# Dataset 2: Home Loan Approval
d2data = {
    'Monthly Salary (₹)': [90000, 80000, 70000, 60000, 50000, 45000, 40000,
        75000, 35000, 95000],
    'CIBIL Score': [850, 820, 780, 740, 690, 670, 640, 800, 620, 870],
    'Existing EMI (₹)': [8000, 9000, 12000, 15000, 18000, 20000, 23000, 10000,
        25000, 7000],
    'Loan Status': ['Approved', 'Approved', 'Approved', 'Approved', 'Rejected',
                    'Rejected', 'Rejected', 'Approved', 'Rejected', 'Approved']
}
```

```
}  
d2sample pd.DataFrame([[72000, 790, 11000]], columns['Monthly Salary (₹)',  
'CIBIL Score', 'Existing EMI (₹)'])
```

```
# Dataset 3: Business Loan Approval
```

```
d3data {  
    'Business Income (₹ Lakhs)': [20, 18, 15, 12, 10, 8, 6, 16, 5, 22],  
    'Business Age (Years)': [10, 9, 8, 6, 5, 4, 3, 8, 2, 12],  
    'Existing Loans': [1, 1, 2, 2, 3, 3, 4, 2, 5, 1],  
    'Loan Status': ['Approved', 'Approved', 'Approved', 'Approved', 'Rejected',  
                    'Rejected', 'Rejected', 'Approved', 'Rejected', 'Approved']  
}  
d3sample pd.DataFrame([[17, 8, 2]], columns['Business Income (₹ Lakhs)',  
'Business Age (Years)', 'Existing Loans'])
```

```
# Dataset 4: Education Loan Approval
```

```
d4data {  
    'Family Income (₹ Lakhs)': [15, 13, 11, 9, 7, 6, 5, 12, 4, 16],  
    'Academic Score (%)': [92, 89, 85, 80, 72, 68, 65, 87, 60, 94],  
    'Loan Amount (₹ Lakhs)': [6, 7, 8, 9, 10, 11, 12, 7, 13, 5],  
    'Loan Status': ['Approved', 'Approved', 'Approved', 'Approved', 'Rejected',  
                    'Rejected', 'Rejected', 'Approved', 'Rejected', 'Approved']  
}  
d4sample pd.DataFrame([[12, 88, 7]], columns['Family Income (₹ Lakhs)',  
'Academic Score (%)', 'Loan Amount (₹ Lakhs)'])
```

```
# Dataset 5: Vehicle Loan Approval
```

```
d5data {  
    'Annual Income (₹ Lakhs)': [14, 12, 10, 8, 7, 6, 5, 11, 4, 15],  
    'Down Payment (₹ Lakhs)': [3, 2.5, 2, 1.8, 1.5, 1.2, 1.0, 2.2, 0.8, 3.5],  
    'Credit Score': [830, 810, 780, 740, 690, 660, 630, 790, 600, 850],  
    'Loan Status': ['Approved', 'Approved', 'Approved', 'Approved', 'Rejected',  
                    'Rejected', 'Rejected', 'Approved', 'Rejected', 'Approved']  
}  
d5sample pd.DataFrame([[11, 2.4, 800]], columns['Annual Income (₹ Lakhs)',  
'Down Payment (₹ Lakhs)', 'Credit Score'])
```

```
# Compile all experiments
```

```
experiments [  
    ("Dataset 1: Personal Loan Approval", d1data, 'Loan Status', d1sample),  
    ("Dataset 2: Home Loan Approval", d2data, 'Loan Status', d2sample),  
    ("Dataset 3: Business Loan Approval", d3data, 'Loan Status', d3sample),  
    ("Dataset 4: Education Loan Approval", d4data, 'Loan Status', d4sample),  
    ("Dataset 5: Vehicle Loan Approval", d5data, 'Loan Status', d5sample)  
]
```

```
#  
# PROCEDURE EXECUTION  
#
```

```
for title, data, targetcol, newsample in experiments:
```

```
    print(""" * 60)  
    print(title)  
    print(""" * 60)
```

```
    # Step 2: Create Dataset (Separate X and y)  
    df = pd.DataFrame(data)  
    X = df.drop(columns[targetcol])  
    y = df[targetcol]
```

```
    # Step 3: Split Dataset (70% train, 30% test)  
    Xtrain, Xtest, ytrain, ytest = train_test_split(  
        X, y, test_size=0.3, random_state=42  
    )
```

```
    # Step 1 & 4: Import & Train Naïve Bayes Model  
    model = GaussianNB()  
    model.fit(Xtrain, ytrain)
```

```
    # Evaluate Test Accuracy  
    test_preds = model.predict(Xtest)  
    acc = accuracy_score(ytest, test_preds)  
    print(f"Gaussian Naïve Bayes Model Accuracy: {acc:.2f}")
```

```
    # Step 5 & 6: Predict & Display Result  
    prediction = model.predict(newsample)[0]
```



```
print("\nSample Input Features:")
print(newsample.tostring(indexFalse))
print(f"\nPredicted Loan Status: {prediction}\n")
```

OUTPUT:

Dataset 1: Personal Loan Approval

Gaussian Naïve Bayes Model Accuracy: 1.00

Sample Input Features:

Annual Income (₹ Lakhs)	Credit Score	Loan Amount (₹ Lakhs)
9	770	6

Predicted Loan Status: Approved

Dataset 2: Home Loan Approval

Gaussian Naïve Bayes Model Accuracy: 1.00

Sample Input Features:

Monthly Salary (₹)	CIBIL Score	Existing EMI (₹)
72000	790	11000

Predicted Loan Status: Approved

Dataset 3: Business Loan Approval

Gaussian Naïve Bayes Model Accuracy: 1.00

Sample Input Features:

Business Income (₹ Lakhs)	Business Age (Years)	Existing Loans
17	8	2

Predicted Loan Status: Approved

Dataset 4: Education Loan Approval

Gaussian Naïve Bayes Model Accuracy: 1.00

Sample Input Features:

Family Income (₹ Lakhs)	Academic Score (%)	Loan Amount (₹ Lakhs)
12	88	7

Predicted Loan Status: Approved

Dataset 5: Vehicle Loan Approval

Gaussian Naïve Bayes Model Accuracy: 1.00

Sample Input Features:

Annual Income (₹ Lakhs)	Down Payment (₹ Lakhs)	Credit Score
11	2.4	800

Predicted Loan Status: Approved

20. Implement Future Sales Prediction using a suitable machine learning algorithm

CODE:

```
import os
import warnings

# Suppress environment warnings
os.environ["LOKYMEXCPUCOUNT"] = "4"
warnings.filterwarnings('ignore', category=UserWarning)

import pandas as pd
from sklearn.linear_model import LinearRegression

#
# DATASET DEFINITIONS & NEW SAMPLES
#
```

Dataset 1: Monthly Product Sales

```
d1data {  
    'Advertising Budget (₹ Thousands)': [20, 25, 30, 35, 40, 45, 50, 55, 60, 65],  
    'Number of Salespersons': [2, 2, 3, 3, 4, 4, 5, 5, 6, 6],  
    'Monthly Sales (Units)': [250, 290, 340, 390, 450, 510, 580, 640, 710, 780]  
}  
d1sample    pd.DataFrame([[48, 5]], columns['Advertising Budget (₹  
Thousands)', 'Number of Salespersons'])
```

Dataset 2: Retail Store Sales

```
d2data {  
    'Store Area (sq.ft)': [1000, 1200, 1400, 1600, 1800, 2000, 2200, 2400, 2600,  
2800],  
    'Daily Customers': [120, 150, 180, 220, 260, 300, 340, 380, 420, 460],  
    'Monthly Sales (₹ Lakhs)': [8, 10, 12, 15, 18, 22, 26, 31, 36, 42]  
}  
d2sample    pd.DataFrame([[2100, 320]], columns['Store Area (sq.ft)', 'Daily  
Customers'])
```

Dataset 3: Online Business Sales

```
d3data {  
    'Website Visitors': [1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000,  
5500],  
    'Advertisement Cost (₹)': [5000, 7000, 9000, 11000, 13000, 15000, 17000,  
19000, 21000, 23000],  
    'Sales (Orders)': [120, 180, 250, 320, 400, 490, 590, 700, 820, 950]  
}  
d3sample    pd.DataFrame([[4200, 18000]], columns['Website Visitors',  
'Advertisement Cost (₹)'])
```

Dataset 4: Supermarket Sales Prediction

```
d4data {  
    'Discount (%)': [5, 8, 10, 12, 15, 18, 20, 22, 25, 28],  
    'Customers per Day': [150, 180, 220, 260, 300, 340, 380, 420, 460, 500],  
    'Weekly Sales (₹ Thousands)': [220, 260, 310, 360, 420, 490, 570, 650, 740,  
840]  
}
```

```

d4sample pd.DataFrame([[18, 350]], columns['Discount (%)', 'Customers per
Day'])

# Dataset 5: Automobile Showroom Sales
d5data {
    'Marketing Budget (₹ Thousands)': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Test Drives Conducted': [20, 25, 30, 35, 40, 45, 50, 55, 60, 65],
    'Cars Sold': [8, 10, 13, 17, 22, 28, 35, 43, 52, 62]
}
d5sample pd.DataFrame([[58, 48]], columns['Marketing Budget (₹
Thousands)', 'Test Drives Conducted'])

# Group experiments
experiments [
    ("Dataset 1: Monthly Product Sales", d1data, 'Monthly Sales (Units)',
d1sample),
    ("Dataset 2: Retail Store Sales", d2data, 'Monthly Sales (₹ Lakhs)',
d2sample),
    ("Dataset 3: Online Business Sales", d3data, 'Sales (Orders)', d3sample),
    ("Dataset 4: Supermarket Sales Prediction", d4data, 'Weekly Sales (₹
Thousands)', d4sample),
    ("Dataset 5: Automobile Showroom Sales", d5data, 'Cars Sold', d5sample)
]

#
# PROCEDURE EXECUTION
#

for title, data, targetcol, newsample in experiments:
    print(""" * 65)
    print(title)
    print(""" * 65)

# Step 2: Separate Input Features (X) and Target Values (y)
df pd.DataFrame(data)
X df.drop(columns[targetcol])
y df[targetcol]

```

```

# Step 1 & 3: Import LinearRegression and Train the Model
model LinearRegression()
model.fit(X, y)

# Step 4: Predict Future Sales
prediction model.predict(newsample)[0]

# Step 5: Display the Predicted Result
print("New Input Features:")
print(newsample.tostring(indexFalse))
print(f"\nPredicted Future Sales ({targetcol}): {prediction:.2f}\n")

```

OUTPUT:

Dataset 1: Monthly Product Sales

New Input Features:

Advertising Budget (₹ Thousands)	Number of Salespersons
48	5

Predicted Future Sales (Monthly Sales (Units)): 563.10

Dataset 2: Retail Store Sales

New Input Features:

Store Area (sq.ft)	Daily Customers
2100	320

Predicted Future Sales (Monthly Sales (₹ Lakhs)): 25.29

Dataset 3: Online Business Sales

New Input Features:

Website Visitors	Advertisement Cost (₹)
4200	18000

Predicted Future Sales (Sales (Orders)): 664.49

Dataset 4: Supermarket Sales Prediction

New Input Features:

Discount (%)	Customers per Day
18	350

Predicted Future Sales (Weekly Sales (₹ Thousands)): 535.66

Dataset 5: Automobile Showroom Sales

New Input Features:

Marketing Budget (₹ Thousands)	Test Drives Conducted
58	48

Predicted Future Sales (Cars Sold): 35.60